Capítulo 6 · Seguridad en MongoDB

Cuaderno de prácticas — de una conexión anónima a una identidad con certificado

Nivel de partida: no se supone experiencia previa en seguridad de MongoDB.
Requisitos: Docker con Compose, archivos del repositorio MONGODB y una terminal. Para la ampliación TLS/X.509 necesitarás Bash y OpenSSL.
Datos: exclusivamente documentos ficticios. Nunca uses las operaciones de preparación sobre servidores reales.

Este cuaderno se puede leer de dos formas: primero sigue el **itinerario esencial**, que enseña contraseñas y roles, y después realiza la ampliación de certificados si el equipo está preparado.

Una práctica estará terminada cuando puedas explicar tanto un **éxito** como un **rechazo**. Un error esperado forma parte del aprendizaje, no se considera fracaso del alumno.

---

0. Antes de empezar: reconocer los tres tipos de instrucciones

A. Lo que se escribe en PowerShell o Bash

Órdenes de Docker y del sistema operativo:

```
docker compose ps
docker compose logs --tail=30 mongo-seguridad
```

B. Lo que se escribe dentro de mongosh

Consultas y comandos de MongoDB:

```
db.getName()
db.secretos.findOne()
```

C. Lo que se ejecuta como script

Un fichero `.js` puede guardar varias órdenes de MongoDB. Desde la terminal lo lanzamos con:

```
docker compose exec mongo-seguridad mongosh --file /lab/08_configurar_seguridad.js
```

Atención: este último ejemplo está incompleto a propósito: crear usuarios exige autenticar antes al administrador. Cuando lleguemos a ese paso se mostrarán todas las opciones necesarias. No lo ejecutes todavía.

Tu primera regla de diagnóstico

Cuando un comando falla, identifica **dónde lo ejecutaste**: PowerShell/Bash, mongosh de índices, mongosh de seguridad o mongosh de TLS. Muchos problemas de principiantes son en realidad cambios de terminal, servidor o base de datos.

---

1. Preparar un laboratorio sin publicar una contraseña

Qué haremos

Necesitamos dos instancias diferentes: una para índices sin autenticación, y otra para demostrar que una contraseña y un rol pueden restringir el acceso. No hace falta conocer aún TLS.

Paso 1. Comprobar la carpeta

Descarga o clona el repositorio. Abre una terminal en su directorio principal. Debes ver el archivo `compose.yaml` y la plantilla `.env.example`.

En PowerShell:

```
Get-ChildItem
```

En Bash:

```
ls -la
```

Si no localizas `compose.yaml`, cambia de carpeta antes de continuar. No sirve iniciar Compose desde cualquier directorio.

Paso 2. Crear tu archivo privado de variables

PowerShell:

```
Copy-Item .env.example .env
notepad .env
```

Bash:

```
cp .env.example .env
# Abre .env con el editor instalado en tu equipo
```

El archivo debe contener tres variables de laboratorio:

```
MONGO_ADMIN_PASSWORD=...
LAB_READER_PASSWORD=...
LAB_WRITER_PASSWORD=...
```

Sustituye los tres valores por contraseñas **de prueba, distintas entre sí**, que puedas recuperar durante la sesión. No pegues esas contraseñas en un chat, captura o commit. `.env` está ignorado por Git, pero debes asegurarte de no publicarlo por otros medios.

¿Por qué no copiamos aquí una contraseña fija? Porque enseñar seguridad utilizando una contraseña pública y permanente generaría exactamente el hábito que debemos evitar.

Paso 3. Iniciar la instancia protegida

En la **terminal**:

```
docker compose up -d --wait mongo-seguridad
docker compose ps
```

Busca el servicio `mongo-seguridad` en estado `healthy`. Su puerto externo es `127.0.0.1:27018`. El nombre `mongo-seguridad` se utiliza dentro de Docker Compose, no necesariamente como nombre de host desde tu ordenador.

Paso 4. Comprobar que el servidor responde a nivel técnico

Puedes abrir el intérprete sin contraseña:

```
docker compose exec mongo-seguridad mongosh
```

Dentro de `mongosh`:

```
db.adminCommand({ping: 1})
```

Es posible que devuelva `ok: 1` aun cuando no estés autenticado. Esto **no implica permiso para leer colecciones privadas**. El comando `ping` se usa para comprobar disponibilidad, no para demostrar privilegios sobre datos.

Sal con `exit`.

Si falla el arranque

Prueba `docker compose logs --tail=80 mongo-seguridad`, confirma que la variable `MONGO_ADMIN_PASSWORD` existe en `.env` y revisa si el puerto 27018 está ocupado. Si cambias una contraseña de inicialización después de haber creado el volumen, MongoDB no necesariamente recreará el usuario: **modificar `.env` no es equivalente a cambiar una contraseña existente dentro de MongoDB**. En un laboratorio desechable se puede reiniciar el entorno, pero evita borrar volúmenes sin entender qué datos perderías.

---

2. Primera prueba negativa: intentar leer sin identidad

Qué pregunta responde esta prueba

¿Puede un visitante que llega al servidor consultar la colección de información protegida sin aportar credenciales?

Paso 1. Conectar anónimamente

En la **terminal**:

```
docker compose exec mongo-seguridad mongosh
```

Paso 2. Elegir la base de datos

Dentro de `mongosh`:

```
use curso_seguridad
```

La orden `use` selecciona una base. **No concede permisos** ni garantiza que la base contenga documentos. Es una elección de contexto.

Paso 3. Intentar una lectura

```
db.secretos.findOne()
```

Esperamos una denegación por falta de autorización. Si la colección todavía no contiene documentos, el comportamiento exacto puede resultar confuso para interpretar; por esa razón, en la siguiente práctica prepararemos datos de demostración como administrador y repetiremos esta prueba.

Paso 4. Interpretar el error

No escribas simplemente «no funciona». Anota:

- Te has conectado al servicio `mongo-seguridad`.
- No has proporcionado usuario ni contraseña.
- Has intentado una operación de lectura sobre datos.
- El servidor ha rechazado la operación por falta de autorización.

Lo que hemos aprendido: poder conectarse a un puerto no concede por sí solo derecho a leer datos.

Si la operación devuelve documentos

Comprueba que no estés conectado a `mongo-indices`, que usa intencionadamente una configuración distinta. También comprueba con `docker compose ps` qué servicio tiene el puerto 27018. Una lectura anónima exitosa de datos protegidos sería un hallazgo a investigar, no una razón para dar la práctica por aprobada.

---

3. Conectarse como administrador y crear datos de ejemplo

Por qué necesitamos un administrador

Los usuarios no deben poder asignarse a sí mismos privilegios que no tienen. Para preparar cuentas docentes utilizaremos temporalmente el administrador inicial, denominado `adminlab`, cuya contraseña configuraste en `.env`.

Paso 1. Abrir mongosh autenticado

Sal de cualquier sesión anterior. En la **terminal**:

```
docker compose exec mongo-seguridad mongosh --username adminlab --authenticationDatabase admin --
password
```

El programa solicitará la contraseña de `MONGO_ADMIN_PASSWORD`. Al escribirla puede que no aparezcan caracteres en pantalla. Eso es normal; evita que alguien la lea por encima de tu hombro.

¿Qué significa cada opción?

- `--username adminlab`: quién intenta identificarse.
- `--authenticationDatabase admin`: dónde está registrada esa cuenta.
- `--password`: solicita la contraseña de manera interactiva.

Paso 2. Probar acceso administrativo a la base del curso

Dentro de `mongosh`:

```
use curso_seguridad
db.secretos.insertOne({
  _id: "demo-1",
  texto: "registro autorizado",
  clasificacion: "docente"
})
```

Deberías recibir confirmación de inserción. Si ya existe el documento `demo-1`, obtendrás un error `E11000`: significa que el ejemplo estaba creado previamente. En ese caso no necesitas duplicarlo.

Paso 3. Leer el mismo registro

```
db.secretos.findOne({_id: "demo-1"})
```

Deberías ver el texto y la clasificación guardados. Esto confirma que el dato existe y que el administrador dispone de permisos suficientes.

Paso 4. Repetir la lectura anónima

Sal del intérprete, vuelve a conectarte con `docker compose exec mongo-seguridad mongosh` y repite:

```
use curso_seguridad
db.secretos.findOne({_id: "demo-1"})
```

Ahora la comparación es más sólida: sabemos que el documento existe y que el administrador puede leerlo; el visitante anónimo no debería poder hacerlo.

Cuestión de comprensión

¿Hemos demostrado que el administrador tiene buena contraseña? Hemos demostrado que **las credenciales suministradas son aceptadas** y que esa cuenta tiene los permisos necesarios. La fortaleza de la contraseña depende de otras políticas.

---

4. Crear un lector y un escritor con roles diferentes

El escenario

Atención al cliente debe consultar compras sin cambiarlas. Administración comercial puede crear documentos de prueba. Los representaremos mediante dos cuentas:

Cuenta	Base de autenticación	Rol
`lectorlab`	`curso_seguridad`	`read`
`escritorlab`	`curso_seguridad`	`readWrite`

Las contraseñas provienen de `.env` y no aparecerán escritas en las guías.

Paso 1. Abrir como administrador

En la **terminal**:

```
docker compose exec mongo-seguridad mongosh --username adminlab --authenticationDatabase admin --
password
```

Introduce la clave cuando sea solicitada.

Paso 2. Crear los usuarios mediante el script preparado

Sal de `mongosh` con `exit` y, desde la **terminal**, ejecuta:

```
docker compose exec mongo-seguridad mongosh --username adminlab --authenticationDatabase admin --
file /lab/08_configurar_seguridad.js --password
```

La opción `--file` carga instrucciones guardadas en el repositorio. El script prepara la colección `curso_seguridad.secretos` y crea `lectorlab` y `escritorlab` con contraseñas tomadas del entorno del contenedor.

No es necesario copiar o modificar internamente el script para aprender el concepto: los permisos se observan en los pasos siguientes. Pero sí debes saber qué operación realiza: **crear identidades y asociar roles**.

Paso 3. Verificar que existen las cuentas

Abre de nuevo `mongosh` como `adminlab` y ejecuta:

```
use curso_seguridad
db.getUsers()
```

En la salida deben aparecer `lectorlab` y `escritorlab`. Observa los roles de cada uno.

Qué podría salir mal

Si ves «faltan claves efímeras», el contenedor no ha recibido `LAB_READER_PASSWORD` o `LAB_WRITER_PASSWORD`. Revisa el archivo `.env` y los valores de entorno del servicio en `compose.yaml`, sin publicarlos en logs. Si cambiaste el archivo `.env` **después** de iniciar el contenedor, quizá necesites recrearlo con su configuración actualizada.

Idea importante

Un usuario pertenece a un ámbito de autenticación. Decir «lectorlab está en curso_seguridad» no significa que esté autorizado a leer cualquier base. Su rol está asociado a un recurso determinado.

---

5. Una lectura que DEBE funcionar

Paso 1. Salir de la cuenta administradora

Dentro de `mongosh`:

```
exit
```

Es importante: si sigues como `adminlab`, toda la prueba carece de valor para enseñar mínimo privilegio.

Paso 2. Iniciar sesión como lector

En la **terminal**:

```
docker compose exec mongo-seguridad mongosh --username lectorlab --authenticationDatabase
curso_seguridad --password
```

Introduce `LAB_READER_PASSWORD`, la contraseña que elegiste para el lector, no la del administrador.

Paso 3. Ejecutar la lectura

Dentro de `mongosh`:

```
use curso_seguridad
db.secretos.findOne({_id: "demo-1"})
```

Resultado esperado: aparece el documento de demostración. Esto confirma que la cuenta pudo autenticarse y está autorizada a realizar esa lectura.

Paso 4. Confirmar con otra operación

```
db.secretos.countDocuments({})
```

Debería mostrar cuántos registros existen; puede haber más de uno si realizaste inserciones anteriores. El resultado no tiene que ser un número fijo.

Explicación

El rol `read` permite al lector consultar datos de su base de datos. No le concede por ello permiso para modificar esos datos. Para comprobar esa segunda condición necesitamos la próxima práctica.

---

6. Una escritura que DEBE fallar

Mantén abierta la sesión como `lectorlab`.

Paso 1. Intentar insertar

Dentro de `mongosh`:

```
db.secretos.insertOne({
  _id: "intento-lector",
  texto: "un lector intenta escribir"
})
```

Resultado esperado: un error de autorización. No intentes corregirlo añadiendo el rol `root` al lector: el error confirma exactamente la política que queremos practicar.

Paso 2. Comprobar que el registro NO apareció

```
db.secretos.findOne({_id: "intento-lector"})
```

Debería devolver `null`. ¿Por qué puede leer si no pudo insertar? Porque lectura y escritura son acciones diferentes, y su rol solo autoriza lectura.

Paso 3. Probar también una actualización

```
db.secretos.updateOne(
  {_id: "demo-1"},
  {$set: {texto: "modificado por el lector"}}
)
```

Esperamos un rechazo por permisos. El hecho de que el lector conozca el identificador del documento no le permite cambiarlo.

Paso 4. Comprobar que el dato inicial se conserva

```
db.secretos.findOne({_id: "demo-1"})
```

El texto debería seguir siendo el que creó la preparación del laboratorio.

Qué escribir en el cuaderno

«He iniciado sesión como `lectorlab`. La lectura devuelve el documento, pero la inserción y la actualización se rechazan. Por tanto, la autenticación tuvo éxito y la autorización impidió operaciones no incluidas en el rol `read`».

Si la escritura funciona: verifica con qué usuario estás conectado. Es posible que sigas como administrador. Comprueba también los roles concedidos a `lectorlab` mediante una sesión administrativa.

---

7. La misma inserción SÍ debe funcionar con readWrite

Paso 1. Salir del lector

```
exit
```

Paso 2. Entrar como escritor

En la **terminal**:

```
docker compose exec mongo-seguridad mongosh --username escritorlab --authenticationDatabase
curso_seguridad --password
```

Introduce la contraseña de `LAB_WRITER_PASSWORD`.

Paso 3. Insertar un documento nuevo

Dentro de `mongosh`:

```
use curso_seguridad
db.secretos.insertOne({
  _id: "permitido-escritor",
  texto: "insertado por una cuenta readWrite"
})
```

Esperamos una confirmación de inserción. Si ya habías ejecutado el ejercicio y el identificador existe, usa otro `_id` de demostración o elimina ese documento **solo si estás autorizado y comprendes el cambio**.

Paso 4. Leer el documento insertado

```
db.secretos.findOne({_id: "permitido-escritor"})
```

Debe aparecer el documento creado.

Paso 5. Explicar las tres pruebas

Conexión	Consulta	Inserción	Razón
Sin credenciales	Rechazada	No corresponde	Falta autenticación
Lector autenticado	Permitida	Rechazada	Rol `read`
Escritor autenticado	Permitida	Permitida	Rol `readWrite`

¿Por qué no basta probar la escritura como escritor? Porque debemos observar también que el lector no consigue realizarla. La seguridad se demuestra contrastando el resultado permitido con el prohibido.

---

8. Contraseña errónea y base de autenticación equivocada

Prueba A. Contraseña incorrecta

Sal de `mongosh`. En la **terminal**:

```
docker compose exec mongo-seguridad mongosh --username lectorlab --authenticationDatabase
curso_seguridad --password
```

Cuando el programa solicite la contraseña, introduce voluntariamente una clave que **no** coincida con `LAB_READER_PASSWORD`. Esperamos un error de autenticación.

Ahora repite con la contraseña correcta. La conexión debe funcionar.

Prueba B. Autenticación contra la base equivocada

`lectorlab` se ha definido en `curso_seguridad`. Intenta conectarte indicando `admin` como base de autenticación:

```
docker compose exec mongo-seguridad mongosh --username lectorlab --authenticationDatabase admin --
password
```

Aunque introduzcas la contraseña de lector correcta, esperamos que el servidor rechace este intento si no existe otra cuenta con ese nombre en `admin`.

Conclusión

La autenticación no depende solo de una pareja texto-contraseña; también necesita la **base de autenticación correcta**. En cadenas de conexión reales esto se expresa con `authSource`.

Si los dos intentos funcionan

Comprueba que realmente has escrito un `authenticationDatabase` distinto en cada caso y que no existe una segunda cuenta del mismo nombre en `admin`. No modifiques permisos para forzar un resultado; comprende primero qué identidad te está autenticando.

---

9. Preparar certificados: qué es una CA y por qué no vale cualquier archivo

Esta sección es **ampliación**. Requiere Bash y OpenSSL además de Docker; en Windows puedes utilizar Git Bash o WSL cuando estén correctamente configurados. El itinerario esencial de contraseña y roles ya puede darse por aprendido si has completado las prácticas anteriores.

Paso 1. Abrir una terminal con Bash y OpenSSL

Comprueba:

```
openssl version
docker compose version
```

Si no puedes ejecutar `openssl`, no improvises certificados copiando archivos ajenos. Puedes estudiar esta sección y revisar el resultado del laboratorio automático de GitHub, sin presentar la ampliación como ejecutada localmente.

Paso 2. Generar certificados temporales de clase

Desde la raíz del repositorio:

```
bash scripts/generar-certificados.sh
```

El script crea una carpeta local `certs/` con varios archivos. Los principales son:

- `ca.crt`: certificado público de la CA docente, que permite verificar firmas.
- `server.pem`: certificado y clave del servidor, que debe custodiarse.
- `client.pem`: certificado y clave del cliente de demostración.

Los archivos se han generado para un entorno efímero. Las claves privadas **no deben subirse a GitHub**, y el generador simplifica ciertos permisos para que los contenedores docentes puedan leerlas. No lo uses como procedimiento de custodia de certificados de producción.

Paso 3. Revisar el certificado del servidor

```
openssl x509 -in certs/server.crt -noout -subject -issuer -dates
```

Busca el sujeto, el emisor y las fechas de validez. No necesitas memorizar el formato: el propósito es reconocer quién figura como identidad del certificado, quién lo firmó y cuándo expira.

Paso 4. Comprobar la cadena de confianza

```
openssl verify -CAfile certs/ca.crt certs/server.crt certs/client.crt
```

Esperamos indicaciones de verificación satisfactoria. **Una firma correcta no concede acceso a MongoDB:** solo demuestra una pieza del sistema de confianza.

---

10. TLS: una conexión sin cifrado debe ser rechazada

Paso 1. Arrancar la instancia TLS

Después de generar certificados, en la **terminal**:

```
docker compose up -d --wait mongo-tls
docker compose ps
```

El servicio usa `127.0.0.1:27019` en tu equipo. Dentro del contenedor, MongoDB escucha con TLS obligatorio.

Paso 2. Intentar conexión sin TLS

```
docker compose exec mongo-tls mongosh --host localhost
```

Esta conexión debe fallar porque el servidor exige TLS. El mensaje de error puede variar; apunta la frase relevante que indique una conexión rechazada o que no puede establecerse correctamente.

Paso 3. Conectar utilizando TLS

```
docker compose exec mongo-tls mongosh --host localhost --tls --tlsCAFile /certs/ca.crt
```

Ahora debería establecerse la conexión protegida. Dentro de `mongosh`, ejecuta:

```
db.adminCommand({ping: 1})
```

Puedes ver `ok: 1`. El éxito de `ping` solo comprueba la comunicación, no que hayas conseguido permisos sobre una colección protegida.

Paso 4. Explicar la diferencia

Las dos pruebas apuntan al **mismo servidor**. La primera omite TLS; la segunda utiliza TLS y un certificado de la CA de confianza. Por tanto, la diferencia importante es la política de protección del canal.

Fallos habituales

Si el servicio no arranca, revisa `docker compose logs --tail=80 mongo-tls`. Puede faltar `server.pem` o `ca.crt`. Si aparece un error de nombre de servidor o autoridad desconocida, no añadas opciones para desactivar la validación; comprueba los certificados generados y el hostname utilizado.

---

11. X.509: el certificado del cliente debe identificar a un usuario

El paso conceptual

Ya tenemos TLS. Ahora queremos probar **autenticación de cliente mediante certificado**. Un certificado válido no basta por sí solo: MongoDB necesita conocer la identidad del sujeto del certificado y los roles asignados.

Paso 1. Entender la identidad externa

El certificado docente tiene un sujeto similar a:

```
CN=alumno-x509,OU=Alumnos,O=MongoDBFormacion
```

En MongoDB esa identidad se registra en `$external`. El repositorio proporciona la preparación completa para evitar que un principiante cometa un error de formato al copiar el nombre distinguido.

Paso 2. Ejecutar el recorrido completo preparado

Para la demostración integrada, abre una terminal **Bash**, desde la raíz del repositorio, y carga las variables privadas de tu archivo `.env`:

```
set -a
. ./.env
set +a
bash scripts/probar-seguridad.sh
```

Utiliza este método **solo con tu propio archivo `.env` de confianza** y con valores de laboratorio sencillos. El script ejecuta una secuencia de pruebas de SCRAM, roles, TLS y X.509. Si ya preparaste manualmente usuarios o documentos que tienen el mismo nombre en `mongo-tls`, quizá sea necesario reiniciar esa instancia desechable siguiendo las indicaciones del formador.

Nota para Windows: si utilizas PowerShell sin Bash, no copies las órdenes `set -a` o `. ./.env`: pertenecen a Bash, no a PowerShell. Puedes ejecutar las actividades SCRAM manuales de las secciones 1 a 8 y usar GitHub Actions para observar la secuencia integrada.

Paso 3. Observar qué valida

El script debe comprobar que:

- la conexión sin TLS se rechaza;
- la conexión TLS validando la CA funciona;
- se registra el usuario X.509;
- el cliente con su certificado puede leer los datos autorizados;
- el mismo cliente **no puede insertar** porque solo dispone del rol de lectura.

No te limites a mirar el mensaje final de éxito. Busca en la salida las dos situaciones de X.509: lectura permitida e inserción rechazada.

Paso 4. Explicar por qué no son dos pruebas idénticas

TLS resuelve la seguridad de la comunicación. X.509 resuelve la **identidad del cliente** mediante certificado. Los roles siguen siendo necesarios para decidir qué acciones autoriza MongoDB. Un certificado de lector no convierte a esa cuenta en administradora.

Si el script falla

La causa puede ser un certificado caducado, variables de entorno no cargadas, contenedores antiguos con usuarios ya creados o puertos ocupados. Lee el primer error, revisa los logs del servicio correspondiente y evita desactivar TLS para ocultar el problema.

---

12. Lo que NO realizaremos en MongoDB Community: LDAP y cifrado nativo

LDAP en una organización

Un directorio de empleados puede centralizar identidades y grupos. LDAP es un protocolo asociado a esa función; la integración LDAP autogestionada de MongoDB corresponde a Enterprise y está obsoleta desde MongoDB 8.0.

Actividad de comprensión: dibuja el camino «empleado → directorio → identidad o grupos → MongoDB → rol autorizado». Marca qué sistema gestiona la identidad y cuál comprueba los privilegios. No digas que has comprobado LDAP con nuestro contenedor Community: sería falso.

Cifrado en reposo

Imagina que alguien obtiene una copia física del almacenamiento. TLS no soluciona por sí solo ese problema, porque TLS protege los datos **en tránsito**. Existen otras opciones, como cifrado de disco, cifrado de campos o cifrado nativo del motor (dependiente de edición). No son equivalentes.

Actividad de comprensión: para cada amenaza, elige qué mecanismo sería relevante:

Amenaza	Mecanismo relevante
Alguien intercepta tráfico de red	TLS
Una aplicación pide borrar datos sin permiso	RBAC
Un intruso consigue una copia de disco	Cifrado en reposo y gestión de claves
Se publica una contraseña por error	Rotación de secreto y revisión de accesos

Estas son decisiones de arquitectura; no todas se convierten en un experimento sobre Community.

---

13. Evaluación final: cuatro pruebas, cuatro explicaciones

Realiza o describe las siguientes operaciones, indicando **qué identidad utilizaste** y cuál fue el resultado:

1. Un visitante sin contraseña intenta leer `curso_seguridad.secretos`.
2. `lectorlab`, con su contraseña correcta, lee `demo-1`.
3. `lectorlab` intenta insertar un documento nuevo.
4. `escritorlab` inserta un documento nuevo.

En cada caso contesta:

- ¿Se estableció la conexión?
- ¿Hubo autenticación?
- ¿Existía permiso para esa operación?
- ¿Qué evidencia observaste?
- ¿Qué cambiaría si eliminaras por error una de las protecciones?

Solución razonada

Visitante anónimo. Puede llegar técnicamente al servicio y quizá ejecutar comandos de disponibilidad, pero no posee autorización para leer datos protegidos.

Lector que consulta. Presenta credenciales correctas y dispone del rol `read` sobre la base prevista; la lectura se permite.

Lector que inserta. La identidad sigue siendo correcta, pero el rol no incluye escritura; la denegación protege los datos.

Escritor que inserta. La cuenta `escritorlab` tiene `readWrite` en la base correspondiente; la inserción se permite.

Si tus resultados son diferentes, **no ajustes las respuestas para hacerlas coincidir con esta solución**. Investiga servidor, base, identidad, contraseña, rol y comando antes de continuar.

---

14. Confirmar con qué identidad estás trabajando

En seguridad es fácil probar accidentalmente una operación como administrador y concluir que el lector tiene permisos. Vamos a verificar el contexto de la sesión antes de interpretar cualquier resultado.

Paso 1. Abrir como lector

En la **terminal**:

```
docker compose exec mongo-seguridad mongosh --username lectorlab --authenticationDatabase
curso_seguridad --password
```

Introduce la contraseña del lector.

Paso 2. Preguntar qué conexión reconoce el servidor

Dentro de `mongosh`:

```
db.runCommand({connectionStatus: 1})
```

Busca el apartado de información de autenticación y los roles. La forma exacta de la respuesta depende de la versión; el dato importante es que estás autenticado como `lectorlab` y no como `adminlab`.

Paso 3. Comprobar una lectura conocida

```
use curso_seguridad
db.secretos.findOne({_id: "demo-1"})
```

Esperamos que pueda leer. La pareja «identidad correcta + lectura exitosa» da una base sólida para la siguiente prueba negativa.

Interpretar

No confundas el nombre que aparece en el prompt de `mongosh` con el nombre del usuario autenticado. El prompt puede mostrar la base actual, no la identidad.

Si los roles no son los esperados: sal de la sesión y vuelve a conectarte como el usuario adecuado. Si crees que se han alterado sus privilegios, un administrador debe inspeccionar su definición antes de modificar nada.

---

15. Actualizar sin permiso: una denegación distinta de insertar

Antes comprobamos que el lector no podía insertar. Ahora veremos que el mismo principio protege modificaciones sobre registros existentes.

Paso 1. Identificar un documento que existe

Como `lectorlab`, dentro de `mongosh`:

```
use curso_seguridad
db.secretos.findOne({_id: "demo-1"})
```

Anota el valor del campo `texto`.

Paso 2. Intentar modificarlo

```
db.secretos.updateOne(
  {_id: "demo-1"},
  {$set: {texto: "modificado sin permiso"}}
)
```

Debe producirse una denegación de autorización. Ese rechazo es un **resultado correcto** del ejercicio, no una razón para conceder `root`.

Paso 3. Verificar que no ha cambiado

```
db.secretos.findOne({_id: "demo-1"})
```

El texto inicial debería mantenerse.

Qué has demostrado

El usuario pudo autenticar y leer, pero no obtuvo autorización para `updateOne`. Si el intento hubiera fallado por un error de sintaxis, no estaríamos demostrando permisos; por eso importa revisar el mensaje concreto.

Si se actualiza correctamente: comprueba con `connectionStatus` si realmente trabajas como `lectorlab` y verifica los roles asignados.

---

16. Borrar sin permiso: proteger un registro ya conocido

La palabra «borrar» puede intimidar, pero el laboratorio está diseñado para comprobar un rechazo sin eliminar el dato original.

Paso 1. Verificar qué documento vamos a proteger

Conectado como lector:

```
use curso_seguridad
db.secretos.findOne({_id: "demo-1"})
```

Paso 2. Intentar borrarlo

```
db.secretos.deleteOne({_id: "demo-1"})
```

Esperamos `Unauthorized` u otro error equivalente de permisos. No debería eliminarse el registro.

Paso 3. Confirmar que sigue ahí

```
db.secretos.findOne({_id: "demo-1"})
```

Debe seguir apareciendo.

La pregunta difícil

Si en lugar de un lector utilizáramos una cuenta `readWrite`, ¿podríamos borrar el documento? Es posible, según los permisos asignados y la operación; por eso `readWrite` no debe ofrecerse a procesos que solo necesitan consultar.

Una precaución útil

No copies este experimento sobre una colección empresarial. Aquí usamos un registro ficticio que hemos preparado precisamente para estudiar accesos permitidos y denegados.

---

17. Un rol personalizado que solo permite consultar una colección

Hasta ahora hemos utilizado roles incorporados. Es útil comprender que una organización puede necesitar permisos más específicos: alguien puede requerir lectura de `secretos` sin acceso general a otras colecciones de la base.

Esta práctica es **de ampliación** y modifica la configuración de usuarios de prueba. Trabaja únicamente en la base del curso.

Paso 1. Abrir sesión como administrador

En la **terminal**:

```
docker compose exec mongo-seguridad mongosh --username adminlab --authenticationDatabase admin --
password
```

Paso 2. Crear un rol específico

Dentro de `mongosh`:

```
use curso_seguridad
db.createRole({
  role: "soloConsultarSecretos",
  privileges: [{
    resource: {db: "curso_seguridad", collection: "secretos"},
    actions: ["find"]
  }],
  roles: []
})
```

Qué significa: el rol contiene un privilegio `find` sobre una colección concreta. No otorga automáticamente escritura ni administración.

Si MongoDB indica que el rol ya existe porque repetiste el ejercicio, inspecciónalo antes de intentar crearlo de nuevo. No elimines roles ajenos al laboratorio.

Paso 3. Crear una cuenta de ampliación

En la sesión interactiva como administrador:

```
db.createUser({
  user: "consultorlab",
  pwd: passwordPrompt(),
  roles: [{role: "soloConsultarSecretos", db: "curso_seguridad"}]
})
```

`passwordPrompt()` solicita una contraseña de manera interactiva cuando está disponible en `mongosh`. Usa una contraseña de prueba distinta de las demás y consérvala solo para este laboratorio.

Paso 4. Probar la cuenta

Sal del administrador y conecta como `consultorlab`. Verifica `findOne` en `secretos` y una inserción denegada. No basta comprobar que el comando de creación devolvió éxito; la validez de la política se observa con la identidad afectada.

---

18. Revocar un rol y recuperarlo sin perder el control

Los permisos pueden cambiar. Una persona pierde temporalmente el acceso a ciertas ventas y después lo recupera.

No hagas esta prueba en la cuenta administradora. Utilizaremos `lectorlab` porque es desechable.

Paso 1. Observar el estado inicial

Conecta como `lectorlab` y comprueba que puede leer `demo-1`. Sal después de la sesión.

Paso 2. Revocar como administrador

```
use curso_seguridad
db.revokeRolesFromUser(
  "lectorlab",
  [{role: "read", db: "curso_seguridad"}]
)
```

Ejecuta esto **solo** en una sesión autenticada como `adminlab` y tras comprobar que estás en la base `curso_seguridad`.

Paso 3. Reconectar como lector

Abre una nueva sesión como `lectorlab` e intenta leer `demo-1`. Esperamos que carezca de permisos para ese acceso.

Paso 4. Restaurar el rol

Vuelve a conectarte como administrador y ejecuta:

```
use curso_seguridad
db.grantRolesToUser(
  "lectorlab",
  [{role: "read", db: "curso_seguridad"}]
)
```

Ahora abre de nuevo la sesión del lector y repite `findOne`. Debería permitirse la lectura.

Qué demuestra

Una autorización no es una cualidad permanente de la contraseña. Los permisos efectivos pueden cambiar sin cambiar la identidad de la cuenta.

Si algo falla: confirma qué sesión es administrativa, qué roles tenía el usuario y si la sesión lectora se ha reconectado. No abandones la actividad antes de devolver los permisos al estado previsto para las demás prácticas.

---

19. Una base distinta no debe heredarse por accidente

Una cuenta `read` sobre `curso_seguridad` no se convierte en un lector universal de cualquier base por haber iniciado sesión correctamente.

Paso 1. Abrir como lector

Conecta como `lectorlab` utilizando `--authenticationDatabase curso_seguridad`.

Paso 2. Leer dentro de su ámbito

```
use curso_seguridad
db.secretos.findOne({_id: "demo-1"})
```

Este acceso debe estar permitido.

Paso 3. Cambiar de base lógica

```
use curso_indices
db.ventas.findOne()
```

Estamos conectados al **servidor de seguridad**, no a la instancia de índices. Puede que la colección ni exista allí. No interpretes un resultado `null` como demostración completa de aislamiento: para verificar permisos se necesita una operación y un recurso preparados.

Paso 4. Interpretación correcta

La cuenta tiene roles asociados a una base. El alcance de `read` no se amplía automáticamente cuando ejecutamos `use otra_base`. Un diagnóstico riguroso diferencia la ausencia de datos de una denegación de autorización.

Lo que hemos aprendido: base de autenticación, base seleccionada y ámbito de los roles son tres dimensiones relacionadas pero no idénticas.

---

20. La cadena de conexión: leerla sin dejar credenciales visibles

Las aplicaciones utilizan a menudo URI de conexión. No es necesario que un principiante las memorice, pero sí debe reconocer sus componentes.

Primera lectura

```
mongodb://localhost:27018/?authSource=admin
```

Se trata de un servidor en el equipo local, puerto 27018, y se indica que la autenticación se realizará contra la base `admin`.

Segundo ejemplo

```
mongodb://localhost:27018/curso_seguridad?authSource=curso_seguridad
```

Aquí el contexto inicial y el ámbito de autenticación se refieren a `curso_seguridad`, como ocurre con el lector docente.

Qué no vamos a escribir en el libro

No añadiremos al texto una contraseña permanente como parte de la URI. Una contraseña puede terminar copiada al historial de terminal, una captura o un repositorio público.

Pregunta de comprensión

¿Cambiar `authSource` concede permisos sobre una colección? No. Solo cambia el lugar donde se comprueba la identidad. La autorización sigue dependiendo de los roles y del recurso al que se intenta acceder.

Diagnóstico de un error frecuente

Si las credenciales son correctas pero MongoDB rechaza el login, revisa usuario, contraseña y `authSource`. No cambies primero los roles: quizá el problema ocurre antes de evaluar permisos.

---

21. Inspeccionar certificados sin asumir que están bien

Un archivo con extensión `.crt` no garantiza que se trate de un certificado válido para tu servidor. Antes de usarlo debemos conocer su sujeto, emisor y fechas.

Paso 1. Generar los certificados de laboratorio

Desde una terminal Bash con OpenSSL:

```
bash scripts/generar-certificados.sh
```

Paso 2. Leer el servidor

```
openssl x509 -in certs/server.crt -noout -subject -issuer -dates
```

Identifica:

- `subject`: identidad a la que se refiere el certificado.
- `issuer`: quién lo emitió.
- `notBefore` y `notAfter`: intervalo de validez.

Paso 3. Leer el cliente

```
openssl x509 -in certs/client.crt -noout -subject -issuer -dates
```

Compara los sujetos. El servidor y el alumno tienen propósitos distintos: presentar un certificado de servidor no equivale a identificarse como el alumno X.509.

Paso 4. Verificar la CA

```
openssl verify -CAfile certs/ca.crt certs/server.crt certs/client.crt
```

La validación de firmas es una pieza de la confianza; el servidor necesitará además disponer de la configuración TLS y de una cuenta adecuada para X.509.

---

22. Un nombre de servidor no es decorativo en TLS

Una conexión TLS debe proteger el canal y permitir validar a qué servidor creemos conectarnos. El nombre del host y las identidades admitidas por el certificado deben corresponderse.

Paso 1. Usar el nombre preparado

Desde la terminal:

```
docker compose exec mongo-tls mongosh --host localhost --tls --tlsCAFile /certs/ca.crt
```

El certificado de laboratorio incluye `localhost` como identidad permitida. Si todos los archivos son recientes y correctos, esperamos poder conectar.

Paso 2. Estudiar el problema del nombre equivocado

Imagina que el certificado solo contemplase `mongodb.interno.example`, pero el cliente intentase conectar a `otro-servidor.example`. El cliente no debería tratar ambas identidades como equivalentes sin verificación.

No necesitas configurar dominios externos para entender el problema. Lo importante es la pregunta que hace el cliente: **¿este certificado corresponde realmente al servidor que he solicitado?**

Paso 3. Rechazar la falsa solución

Algunos errores se «arreglan» con opciones que omiten validación de certificados. No las utilices: el objetivo es aprender a identificar un error de confianza, no evitarlo sin comprender las consecuencias.

Conclusión

Cifrar sin verificar adecuadamente a quién pertenece la otra parte puede proteger menos de lo que aparenta. La validación del nombre y de la cadena de confianza es parte esencial de la protección del canal.

---

23. TLS válido, usuario no autorizado: dos comprobaciones distintas

Volvamos al servidor `mongo-tls` después de generar certificados.

Conectar con canal protegido

```
docker compose exec mongo-tls mongosh --host localhost --tls --tlsCAFile /certs/ca.crt
```

Dentro de `mongosh`:

```
db.adminCommand({ping: 1})
```

Podrías obtener `ok:1`. Esto significa que el servidor responde sobre el canal protegido.

Intentar acceder a datos protegidos

```
use curso_cert
db.pruebas.findOne({_id: 1})
```

Si el servidor exige autorización y no has presentado una identidad admitida, la lectura debe rechazarse. **No es una contradicción**: el canal puede estar correctamente protegido aunque no poseas un rol de lectura.

Registrar dos evidencias

Anota la conexión TLS válida y la consulta denegada. Una prueba no sustituye a la otra.

Qué aprenderías con el certificado del alumno

Cuando X.509 identifica al cliente y MongoDB le asigna un rol `read`, esa misma lectura puede permitirse. Todavía debe rechazarse una inserción, porque el certificado no regala permisos de escritura.

---

24. Reconocer la identidad X.509 y su ámbito

La cuenta X.509 de laboratorio se asocia a un sujeto de certificado como:

```
CN=alumno-x509,OU=Alumnos,O=MongoDBFormacion
```

La identidad no es una palabra arbitraria que escribamos al conectarnos; debe corresponder al certificado presentado y al usuario que MongoDB tiene registrado en `$external`.

Paso 1. Reproducir la preparación integrada

Desde Bash en la carpeta del proyecto, con certificados ya generados:

```
set -a
. ./.env
set +a
bash scripts/probar-seguridad.sh
```

Observa los mensajes relacionados con X.509.

Paso 2. Comprobar las dos acciones

El script debe verificar que la cuenta X.509 puede leer `curso_cert.pruebas` y que se le deniega escribir en ella.

Paso 3. Explicar lo ocurrido

No digas simplemente «los certificados funcionan». Di que **un cliente presentó un certificado reconocido, obtuvo una identidad y quedó sujeto a roles de autorización**, igual que sucede con un usuario SCRAM.

Si falla

Revisa primero certificados, CA y estado del contenedor. Después investiga el formato del sujeto registrado. No cambies a `root` la identidad X.509 para evitar estudiar el problema.

---

25. Un intento de escritura fallido NO tiene una sola causa

Dos mensajes de error pueden parecer parecidos, pero originarse en lugares diferentes.

Caso A: contraseña equivocada

El cliente no puede demostrar su identidad. El servidor no llega a tratarlo como el lector previsto.

Caso B: rol de solo lectura

El cliente ya se ha autenticado como `lectorlab`, pero intenta un `insertOne` que su rol no autoriza.

Caso C: TLS obligatorio

La conexión ni siquiera cumple la política de protección del canal. El rechazo ocurre antes de una operación normal de lectura o escritura.

Ejercicio de diagnóstico

Clasifica cada mensaje conceptual:

1. «Authentication failed» durante el login.
2. «not authorized» después de entrar como lector e intentar insertar.
3. Error al establecer un canal con un servidor que exige TLS.

Solución: el primero apunta a autenticación; el segundo a autorización; el tercero al canal de comunicación. Para diagnosticar un caso concreto hay que revisar el comando, el contexto y los logs, no solo buscar una palabra conocida.

Por qué importa

La reacción equivocada sería desactivar la seguridad para que los errores desaparezcan. El objetivo de la seguridad es precisamente bloquear algunas operaciones.

---

26. Revisar logs sin publicar contraseñas

Los registros de Docker ayudan a explicar un servidor que no arranca o una conexión fallida. No sustituyen una auditoría formal completa.

Paso 1. Consultar un fragmento limitado

En la terminal:

```
docker compose logs --tail=40 mongo-seguridad
```

Observa la fecha, el servicio y el mensaje relevante. No pegues todo el log en un documento que también contenga contraseñas privadas.

Paso 2. Distinguir tres clases de problemas

- Un **fallo de arranque** se detecta en el ciclo del contenedor o en el proceso de MongoDB.
- Un **fallo de autenticación** ocurre cuando las credenciales no permiten demostrar identidad.
- Un **rechazo de autorización** puede ser el comportamiento correcto del rol, no una avería del servidor.

Paso 3. Hacer una anotación útil

Escribe: «El servicio estaba sano, el cliente se conectó como lector y la escritura se rechazó por permisos». Esto enseña más que copiar cincuenta líneas sin explicar qué ocurrió.

Precaución

Los mensajes de diagnóstico pueden incluir datos sensibles según el sistema y sus opciones. Antes de compartirlos, revisa que no contengan secretos ni información privada. La práctica utiliza datos ficticios, pero debemos aprender hábitos válidos para producción.

---

27. Actividad final: preparar una política mínima para la librería

La librería tiene tres procesos:

- Pantalla de atención: consulta ventas.
- Servicio de pedidos: registra ventas nuevas.
- Mantenimiento: administra usuarios y configuración.

Primera tarea. Proponer identidades

Asigna a cada proceso una identidad diferente. Explica por qué no deben compartir la cuenta administradora.

Segunda tarea. Proponer permisos

La pantalla de atención necesita `read` sobre los datos que consulta. El servicio de pedidos quizá necesite `readWrite` sobre ciertas colecciones. El mantenimiento requiere funciones administrativas más amplias, pero debe quedar fuera del tráfico ordinario de la aplicación.

Tercera tarea. Proponer protección de canal

Si los clientes se conectan por una red no confiable, explica por qué se necesita TLS. Una contraseña SCRAM sin una política correcta de comunicación no resuelve todas las amenazas.

Cuarta tarea. Proponer comprobaciones

Diseña una prueba que debe funcionar y otra que debe fallar para cada identidad. Por ejemplo, la pantalla de atención debe leer y no debe borrar.

Quinta tarea. Explicar límites

¿Un rol `read` protege automáticamente un disco robado? No. ¿Un certificado X.509 impide por sí solo que un usuario escriba? No. ¿LDAP de Enterprise puede simularse activando una opción Community inexistente? No.

Entrega

Elabora una tabla de **identidad, recurso, operación permitida, operación prohibida y evidencia esperada**. Esa tabla demuestra que comprendes la seguridad como decisiones observables y no como una acumulación de comandos.

Guía de resolución de errores

«docker: command not found». Docker no está instalado o la terminal no lo encuentra.

«Cannot connect to daemon». Docker Desktop/Engine no está arrancado.

«required variable MONGO_ADMIN_PASSWORD». Falta `.env` o esa variable en el archivo.

«Authentication failed». Revisa identidad, contraseña y `--authenticationDatabase`. No cambies los roles para corregir un error de contraseña.

«not authorized». Comprueba si la operación que intentaste está prevista por el rol. Puede ser el resultado correcto de la práctica.

«E11000 duplicate key». El documento con ese `_id` ya existe. No es un error de rol; repite con un identificador de demostración nuevo.

«No veo demo-1». El dato todavía no se creó, fue eliminado o has escogido otra base. Prepara nuevamente los datos como administrador.

«Certificate verify failed». Comprueba CA, nombres, validez temporal y ruta de archivos; no suprimas la verificación.

«Port already allocated». Otro programa utiliza el puerto externo configurado; inspecciona y ajusta el entorno de manera consciente.

«No puedo editar el archivo .env». Revisa permisos locales y que estás en la carpeta correcta. No guardes tu contraseña en un archivo de documentación pública.

Cierre del aprendizaje

Al terminar deberías poder decir con tus propias palabras:

«La autenticación demuestra quién soy; la autorización decide qué puedo hacer. Un lector puede iniciar sesión correctamente y, aun así, recibir una denegación válida al intentar escribir. TLS protege la comunicación, pero no concede roles. X.509 puede identificar al cliente con un certificado, y los permisos siguen aplicándose. LDAP y el cifrado nativo en reposo autogestionado no son pruebas que podamos atribuir a nuestro contenedor Community».

Quien puede justificar estas ideas y reproducir las operaciones positivas y negativas ha adquirido una base práctica real de seguridad en MongoDB.

Fuentes de consulta

- MongoDB 8.0 — Seguridad
- Autenticación SCRAM
- Certificados X.509